

Trabajo Práctico 5 - Device drivers

Integrantes

- Santiago Alasia
- Lucia Feiguin
- Elena Monutti

Link del Repositorio

```
https://github.com/SantiagoAlasia/The-Pipeline/tree/main/TP5
```

Introducción

Los Controladores de Dispositivos de Caracteres (Character Device Drivers, CDD) constituyen uno de los mecanismos fundamentales mediante los cuales el sistema operativo Linux permite la comunicación entre el espacio de usuario y los dispositivos de hardware. A través de estos controladores, las aplicaciones pueden acceder a periféricos utilizando una interfaz estándar basada en operaciones de lectura y escritura.

En el presente trabajo práctico se desarrolla un CDD para Linux ejecutándose sobre una **Raspberry Pi Modelo 3B**, capaz de adquirir periódicamente dos señales externas digitales cuyo periodo es de 1 segundo. El controlador realiza el muestreo de las señales con una frecuencia de 20Hz (**cumpliendo con el teorema Nyquist-Shannon) y proporciona una interfaz de acceso mediante un dispositivo de caracteres. Además, permite seleccionar dinámicamente cuál de las dos señales será leída por las aplicaciones de usuario.

Como complemento al controlador, se implementa una aplicación de usuario encargada de obtener los datos desde el dispositivo de caracteres, procesarlos y representarlos gráficamente en función del tiempo. La aplicación también permite seleccionar la señal a visualizar y actualizar el gráfico en consecuencia. Para facilitar la interacción con el sistema y minimizar los requerimientos de hardware, la visualización se realiza mediante una interfaz web accesible desde cualquier navegador.

Todo el desarrollo se lleva a cabo utilizando compilación cruzada (cross-compilation), donde el código fuente es escrito y compilado en una computadora anfitriona, generando binarios compatibles con la arquitectura **ARM** de la **Raspberry Pi**. Posteriormente, estos binarios son transferidos al dispositivo para su ejecución y validación.

Objetivos

Diseñar e implementar un Controlador de Dispositivo de Caracteres (CDD) para Linux que permita adquirir dos señales externas mediante una Raspberry Pi y proveer una interfaz de comunicación con aplicaciones de usuario para su visualización y monitoreo en tiempo real.

Objetivos Específicos

- Comprender el funcionamiento de los controladores de dispositivos de caracteres dentro del kernel de Linux.

- Implementar un módulo del kernel capaz de registrar y administrar un dispositivo de caracteres.
- Configurar y utilizar GPIOs de la Raspberry Pi para la adquisición de señales digitales externas.
- Aplicar técnicas de compilación cruzada para generar binarios compatibles con la arquitectura ARM de la Raspberry Pi.
- Validar el correcto funcionamiento del sistema completo mediante pruebas sobre hardware real.

Desarrollo

Creación del primer módulo del kernel

Como primera aproximación al desarrollo de drivers Linux, se implementó un módulo básico del kernel escrito en lenguaje C.

El módulo implementa dos funciones principales:

- Una función de inicialización ejecutada al cargar el módulo mediante `insmod`.
- Una función de salida ejecutada al remover el módulo mediante `rmmod`.

Estas funciones se registran utilizando las macros `module_init()` y `module_exit()`, las cuales forman parte de la infraestructura estándar para módulos Linux.

Además, se utilizó la función `printk()` para registrar mensajes en el log del kernel, permitiendo verificar el correcto funcionamiento del módulo mediante el comando `dmesg`.

Compilación del módulo

La compilación del módulo se realizó utilizando el sistema de build provisto por el kernel Linux mediante un archivo `Makefile`.

El archivo Makefile utilizado fue:

```
obj-m += tp_driver.o
```

```
all: make -C /lib/modules/$(shelluname -r)/buildM=$(PWD) modules
```

```
clean: make -C /lib/modules/$(shelluname -r)/buildM=$(PWD) clean
```

La directiva `obj-m` indica que el archivo `tp_driver.c` debe compilarse como un módulo del kernel.

La compilación se ejecutó mediante:

```
make
```

Como resultado, el sistema generó el archivo `tp_driver.ko`, correspondiente al módulo compilado listo para ser cargado dinámicamente en el kernel Linux.

```
elemnutti@ubuntu1:~/The-Pipeline/TP5/driver$ make
make -C /lib/modules/6.17.0-35-generic/build M=/home/elemonutti/The-Pipeline/TP5/driver modules
make[1]: Entering directory '/usr/src/linux-headers-6.17.0-35-generic'
make[2]: Entering directory '/home/elemonutti/The-Pipeline/TP5/driver'
]warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
You are using:          gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
CC [M]  tp_driver.o
MODPOST Module.symvers
CC [M]  tp_driver.mod.o
CC [M]  .module-common.o
LD [M]  tp_driver.ko
BTF [M] tp_driver.ko
Skipping BTF generation for tp_driver.ko due to unavailability of vmlinux
make[2]: Leaving directory '/home/elemonutti/The-Pipeline/TP5/driver'
make[1]: Leaving directory '/usr/src/linux-headers-6.17.0-35-generic'
```

Figura 1: Compilación exitosa del módulo del kernel y generación del archivo `tp_driver.ko`.

Inserción del módulo

Una vez compilado el módulo, se procedió a cargarlo dinámicamente en el kernel utilizando el comando:

```
sudo insmod tp_driver.ko
```

La correcta inserción del módulo se verificó mediante:

```
lsmod | grep tp_driver
```

Posteriormente, el módulo fue removido utilizando:

```
sudo rmmod tp_driver
```

Este procedimiento permitió validar el correcto funcionamiento de las funciones de inicialización y salida implementadas en el módulo.

```
elemnutti@ubuntu1:~/The-Pipeline/TP5/driver$ sudo insmod tp_driver.ko
elemnutti@ubuntu1:~/The-Pipeline/TP5/driver$ lsmod | grep tp_driver
tp_driver                12288  0
```

Figura 2: Verificación de la correcta carga del módulo `tp_driver` en el kernel Linux.

Verificación mediante dmesg

Los mensajes generados por el módulo fueron verificados mediante el comando:

```
sudo dmesg | tail
```

A través de este mecanismo fue posible observar los mensajes emitidos por `printk()` durante la carga y descarga del módulo.

Esto permitió confirmar la correcta ejecución de las funciones del módulo dentro del espacio de kernel.

```

elenonutti@ubuntu:~/The-Pipeline/TP5/driver$ sudo dmesg | tail
[ 632.319853] audit: type=1400 audit(1780077891.626:173): apparmor="STATUS" operation="profile_replace" profile="unconfined" name="snap-update-ns.discord" pid=7517 comm="apparmor_parser"
[ 632.347216] audit: type=1400 audit(1780077891.652:174): apparmor="STATUS" operation="profile_replace" profile="unconfined" name="snap.discord.discord" pid=7518 comm="apparmor_parser"
[ 632.680966] audit: type=1400 audit(1780077891.986:175): apparmor="DENIED" operation="mount" class="mount" info="failed flags match" error=-13 profile="snap-update-ns.discord" name="/boot/" pid=7536 comm="snap-update-ns" srcname="/var/lib/snapd/hostfs/boot/" flags="rw, bind"
[ 1234.945740] hrtimer: interrupt took 3269478 ns
[ 2805.118916] tp_driver: loading out-of-tree module taints kernel.
[ 2805.118923] tp_driver: module verification failed: signature and/or required key missing - tainting kernel
[ 2805.119244] tp_driver: modulo cargado correctamente
[ 2855.871810] tp_driver: modulo descargado correctamente
[ 3619.107278] tp_driver: modulo cargado correctamente
[ 3632.986656] tp_driver: modulo descargado correctamente

```

Figura 3: Mensajes del kernel generados durante la carga y descarga del módulo.

Implementación del Character Device Driver

Luego de validar el funcionamiento básico de un módulo del kernel, se avanzó en la implementación de un Character Device Driver (CDD).

Para ello, se incorporó el registro dinámico de números mayor y menor mediante la función:

```
alloc_chrdev_region()
```

Esto permitió que el kernel identificara el dispositivo de caracteres implementado.

Posteriormente, se definió una estructura `file_operations`, utilizada para asociar las operaciones del sistema de archivos con las funciones implementadas por el driver. En esta etapa se incorporaron las operaciones:

- `open`
- `read`
- `release`

La estructura utilizada fue:

```
static struct file_operations fops = { .owner = THIS_MODULE, .open = my_open, .read = my_read, .release = my_release, };
```

Además, se utilizó la estructura `cdev` para registrar el dispositivo dentro del kernel mediante:

```
cdev_init() cdev_add()
```

Finalmente, se creó automáticamente el archivo de dispositivo dentro del directorio `/dev` utilizando:

```
class_create() device_create()
```

Como resultado, el sistema generó el archivo:

```
/dev/tp_driver
```

permitiendo la interacción entre espacio de usuario y espacio de kernel mediante operaciones estándar de lectura.

Implementación de la operación `read()`

Se implementó la operación `read()` del driver con el objetivo de transferir información desde el espacio de kernel hacia el espacio de usuario.

Para realizar esta transferencia se utilizó la función:

`copy_to_user()`

la cual permite copiar datos de manera segura desde memoria del kernel hacia un buffer perteneciente a una aplicación de usuario.

La función implementada devuelve inicialmente un mensaje de prueba:

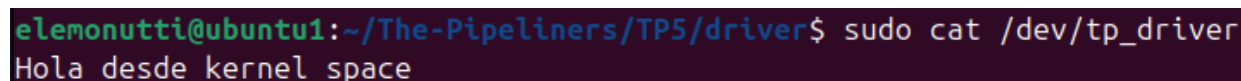
Hola desde kernel space

La operación fue validada mediante el comando:

```
sudo cat /dev/tp_driver
```

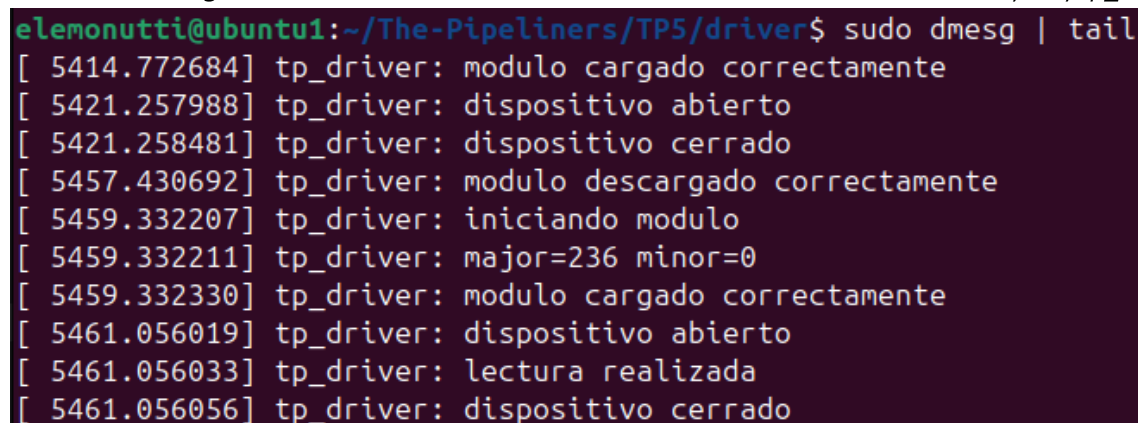
obteniéndose correctamente el mensaje enviado desde el driver.

Además, se implementó el manejo del offset de lectura para evitar lecturas infinitas al utilizar herramientas como `cat`.



```
e1emonutti@ubuntu1:~/The-Pipeline/TP5/driver$ sudo cat /dev/tp_driver
Hola desde kernel space
```

Figura 4: Lectura exitosa desde el Character Device Driver mediante `/dev/tp_driver`.



```
e1emonutti@ubuntu1:~/The-Pipeline/TP5/driver$ sudo dmesg | tail
[ 5414.772684] tp_driver: modulo cargado correctamente
[ 5421.257988] tp_driver: dispositivo abierto
[ 5421.258481] tp_driver: dispositivo cerrado
[ 5457.430692] tp_driver: modulo descargado correctamente
[ 5459.332207] tp_driver: iniciando modulo
[ 5459.332211] tp_driver: major=236 minor=0
[ 5459.332330] tp_driver: modulo cargado correctamente
[ 5461.056019] tp_driver: dispositivo abierto
[ 5461.056033] tp_driver: lectura realizada
[ 5461.056056] tp_driver: dispositivo cerrado
```

Figura 5: Mensajes del kernel que muestran la carga del módulo, la creación del Character Device Driver y la ejecución de las operaciones `open`, `read` y `release`.

Implementación de la operación `write()`

Con el objetivo de permitir que una aplicación de usuario envíe comandos al driver, se implementó la operación `write()` dentro de la estructura `file_operations`.

La transferencia de datos desde espacio de usuario hacia espacio de kernel se realizó mediante la función `copy_from_user()`, la cual permite copiar información de forma segura desde un buffer perteneciente a una aplicación hacia memoria del kernel.

En esta etapa se incorporó una variable interna denominada `sensor_actual`, utilizada para almacenar la señal seleccionada por el usuario. Mediante comandos de escritura sobre el archivo `/dev/tp_driver`, el driver puede recibir el identificador del sensor que deberá utilizar en futuras lecturas.

Esta funcionalidad constituye la base del mecanismo requerido por la consigna para permitir que la aplicación de usuario seleccione cuál de las señales disponibles desea visualizar.

```
elemonutti@ubuntu1:~/The-Pipeline/TP5/CDD$ echo 1 | sudo tee /dev/tp_driver
1
elemonutti@ubuntu1:~/The-Pipeline/TP5/CDD$ echo 2 | sudo tee /dev/tp_driver
2
elemonutti@ubuntu1:~/The-Pipeline/TP5/CDD$ sudo dmesg | tail
[ 1995.474164] tp_driver: module verification failed: signature and/or required
key missing - tainting kernel
[ 1995.474444] tp_driver: iniciando modulo
[ 1995.474447] tp_driver: major=236 minor=0
[ 1995.474854] tp_driver: modulo cargado correctamente
[ 2002.445750] tp_driver: dispositivo abierto
[ 2002.445839] tp_driver: sensor seleccionado = 1
[ 2002.445842] tp_driver: dispositivo cerrado
[ 2006.873962] tp_driver: dispositivo abierto
[ 2006.873976] tp_driver: sensor seleccionado = 2
[ 2006.873979] tp_driver: dispositivo cerrado
```

Figura 6: Escritura de comandos sobre /dev/tp_driver para seleccionar la señal a utilizar por el driver.

Implementación del CDD con GPIO y Timer

Tras validar el correcto funcionamiento de un módulo básico de pruebas en el espacio de kernel, se procedió al diseño y desarrollo del driver definitivo. Sin embargo, como instancia previa a la programación del hardware, resultó necesario preparar el entorno de **desarrollo cruzado** para garantizar un flujo de trabajo eficiente.

Proceso de Compilación Cruzada (Cross-Compilación)

Para la construcción del módulo de kernel se implementó un flujo de compilación cruzada (cross-compilación). Este proceso consiste en utilizar una máquina anfitriona (Host con arquitectura x86_64) con alta capacidad de procesamiento para generar un binario ejecutable final destinado a una arquitectura de hardware completamente diferente (Target con arquitectura ARM64 de 64 bits).

Para que la **cross-compilación** fuera exitosa, se procedió a configurar un entorno de desarrollo integrado que coordinó tres variables esenciales a través del sistema de construcción de Linux (Kbuild). En primer lugar, se instaló y declaró el toolchain específico mediante la bandera **CROSS_COMPILE=aarch64-linux-gnu-**, forzando al sistema a invocar las herramientas de traducción de instrucciones para **ARM64**. En segundo lugar, se configuró la variable **ARCH=arm64** para empaquetar el módulo bajo las reglas de dicha arquitectura. Finalmente, el Makefile se enlazó de forma directa con las cabeceras y el código fuente clonado de la versión exacta del núcleo de la placa destino (rama **Linux Raspberry Pi 6.1.8**), garantizando que las estructuras de datos y macros del kernel coincidan al momento de la inserción dinámica con **insmod**.

Implementación de Arquitectura GPIO Moderna (gpiod)

La vinculación del hardware no se realiza de forma estática en el código de C, sino que se delega al Device Tree Overlay (gpio_overlay.dts). Este archivo define un nodo compatible con la firma "the-pipeline,gpio-driver", reservando los pines físicos GPIO 20 y GPIO 21 bajo los alias lógicos "sensor1" y "sensor2". Cuando el kernel detecta la carga de este overlay, dispara la función de inicialización **probe** del driver de plataforma. Dentro de esta rutina, el módulo utiliza la API segura **devm_gpiod_get()** para solicitar los descriptores de las líneas configuradas como entradas (GPIOD_IN). Este enfoque no solo blindo el acceso al hardware

aislando los pines de otros procesos, sino que permite conmutar de forma segura entre canales en tiempo de ejecución escribiendo un carácter ('1' o '2') en `/dev/gpio_driver`.

Muestreo Periódico Mediante Temporizadores de Kernel (Timers)

Para recolectar el nivel lógico de los sensores sin usar un **polling** continuo que degradaría críticamente el rendimiento del sistema operativo, se implementó un timer del kernel mediante la estructura `struct timer_list`.

Durante la fase de inicialización (**probe**), el driver configura y acopla este temporizador interno a una función de **callback** dedicada encargada de realizar la lectura física del hardware de forma automatizada. El temporizador se calibra utilizando la variable del sistema `jiffies` para dispararse con una periodicidad controlada. Cada vez que expira el ciclo de tiempo establecido, la función **callback** interroga al descriptor GPIO activo mediante la llamada no bloqueante `gpiod_get_value()`, guarda el estado digital en una variable global segura para su posterior lectura desde el espacio de usuario (`copy_to_user`), y vuelve a programar el temporizador de manera recursiva empleando la macro `mod_timer()`. Este diseño asíncrono asegura un muestreo predecible, determinista y de bajo impacto para el planificador de tareas de Linux.

Desarrollo de la Interfaz Web

La capa de aplicación fue diseñada siguiendo una arquitectura que separa claramente la adquisición de datos de la lógica de visualización. El componente principal de esta capa es el archivo `app.py`, encargado de actuar como intermediario entre el driver de caracteres y la interfaz web. Su funcionamiento se divide en dos responsabilidades fundamentales:

- *Adquisición de datos:* La adquisición de muestras se realiza mediante un hilo de ejecución secundario (**acquisition_loop**) que opera de forma independiente del servidor web. Este hilo efectúa lecturas periódicas sobre el dispositivo de caracteres (`/dev/gpio_driver`) utilizando operaciones no bloqueantes, con una frecuencia aproximada de 20 Hz.

Las muestras obtenidas son almacenadas en memoria dentro de una estructura que funciona como un búfer circular de hasta 400 elementos. Esta estrategia permite conservar únicamente las mediciones más recientes, limitando el consumo de memoria y garantizando una actualización eficiente de los datos mostrados al usuario.

- *Comunicación entre cliente y servidor:* Para permitir la interacción con la interfaz web se implementó una API REST sencilla compuesta por dos rutas principales:
- **/data:** proporciona al navegador las últimas muestras adquiridas junto con la información de estado necesaria para la visualización. Los datos son serializados en formato JSON, facilitando su procesamiento mediante JavaScript en el lado del cliente.
- **/signal:** recibe solicitudes HTTP de tipo POST generadas por el usuario cuando desea cambiar la señal monitoreada. La aplicación traduce estas solicitudes en operaciones de escritura sobre el dispositivo de caracteres, permitiendo que el controlador seleccione dinámicamente cuál de las entradas GPIO será utilizada para la adquisición de datos.

Visualización de las señales

El gráfico utiliza un eje temporal fijo que muestra una ventana con referencias temporales relativas que van desde **-3 s** hasta el instante actual, identificado como **"Ahora"**.

La aplicación actualiza periódicamente el gráfico utilizando las muestras más recientes almacenadas en el búfer. De esta manera, la señal se desplaza de forma continua a lo largo de la pantalla, permitiendo observar su evolución en tiempo real sin necesidad de realizar desplazamientos manuales. Además, cuando el usuario selecciona una señal diferente, la visualización se reinicia automáticamente para evitar mezclar datos pertenecientes a distintas fuentes y garantizar una representación coherente de la nueva medición.

En las siguientes figuras podemos ver como quedo la interfaz web para cada una de las señales.

Señal 1

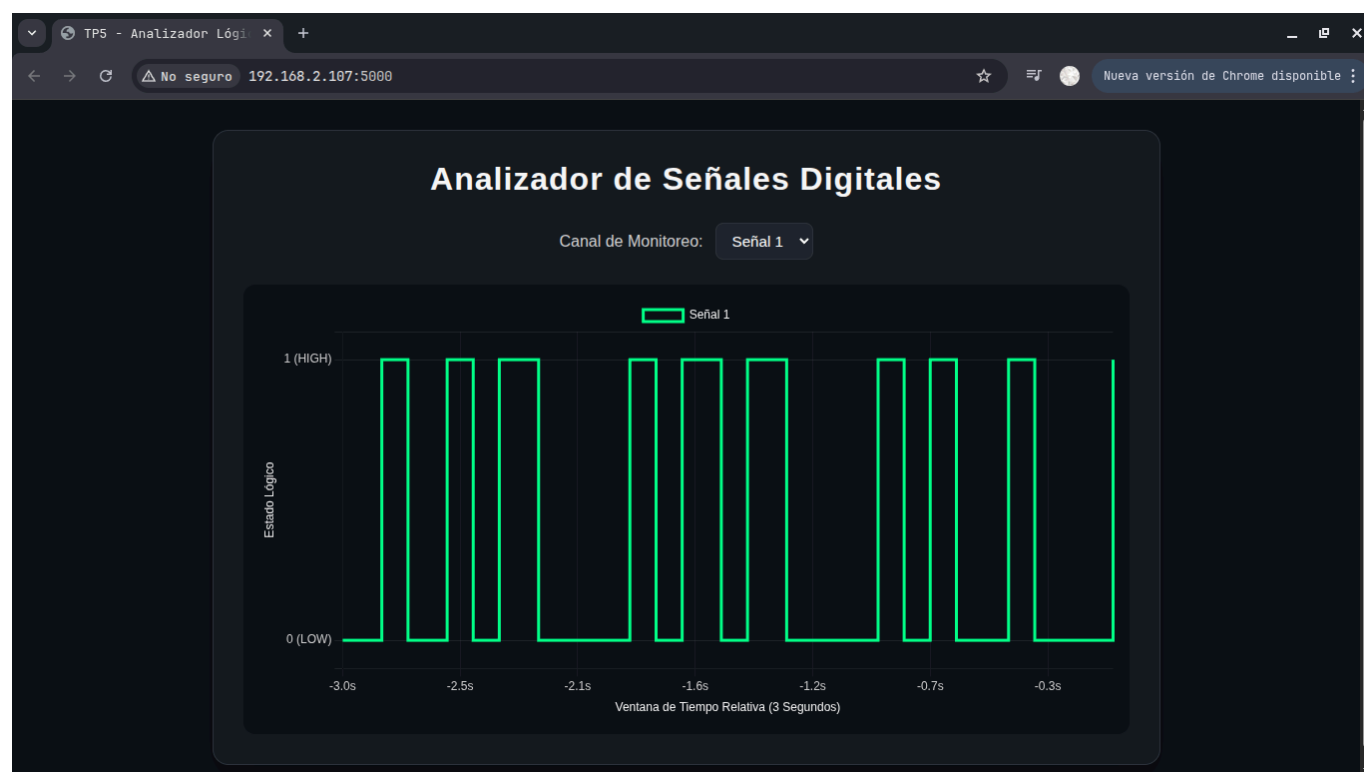


Figura 7. Ventana correspondiente a la señal 1

Señal 2



Figura 8. Ventana correspondiente a la Señal 2.

Creación de un generador de señales para testing

Con el objetivo de validar el correcto funcionamiento del controlador desarrollado y de la aplicación de visualización, se implementó un generador de señales utilizando una placa **Arduino UNO**. Este generador permite producir de forma controlada dos señales digitales periódicas independientes, las cuales son enviadas a los GPIO de la Raspberry Pi para su posterior adquisición por parte del driver.

La utilización de un generador dedicado resultó especialmente útil durante las etapas de desarrollo y depuración, ya que permitió disponer de señales conocidas y repetibles, facilitando la verificación del comportamiento del sistema ante distintos patrones de entrada.

El programa desarrollado para el Arduino genera dos señales digitales periódicas a través de dos pines de salida distintos. Ambas señales poseen un período de un segundo, cumpliendo con los requisitos establecidos en el enunciado del trabajo práctico. Sin embargo, cada una presenta una forma de onda diferente, lo que permite comprobar que el sistema es capaz de distinguir correctamente entre ambas señales y actualizar la visualización cuando el usuario selecciona una u otra.

La forma temporal de las señales generadas se muestra a continuación:

Señal 1

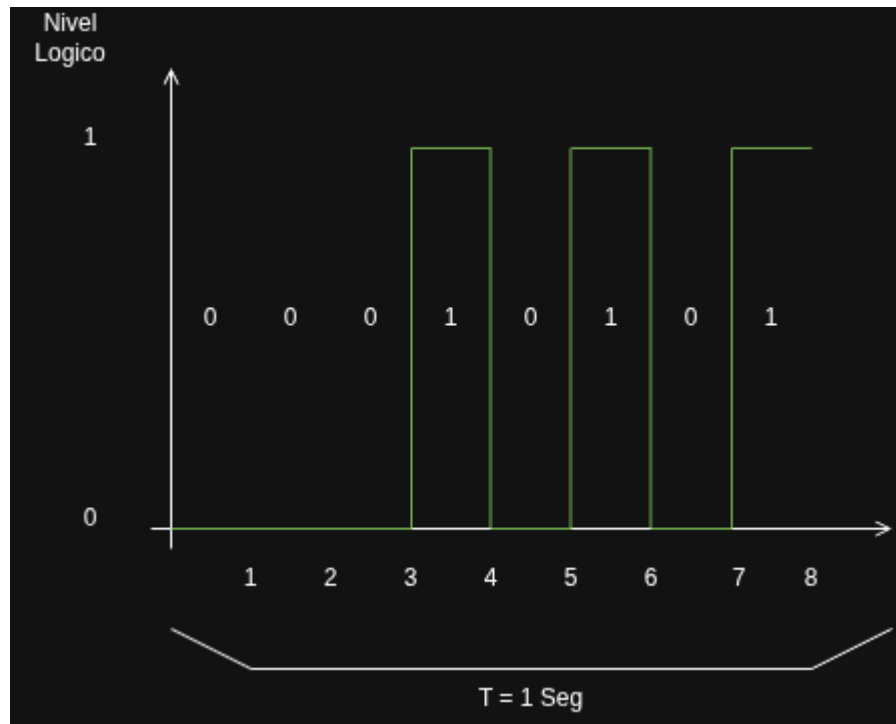


Figura 9. Forma de onda correspondiente a la Señal 1.

Señal 2

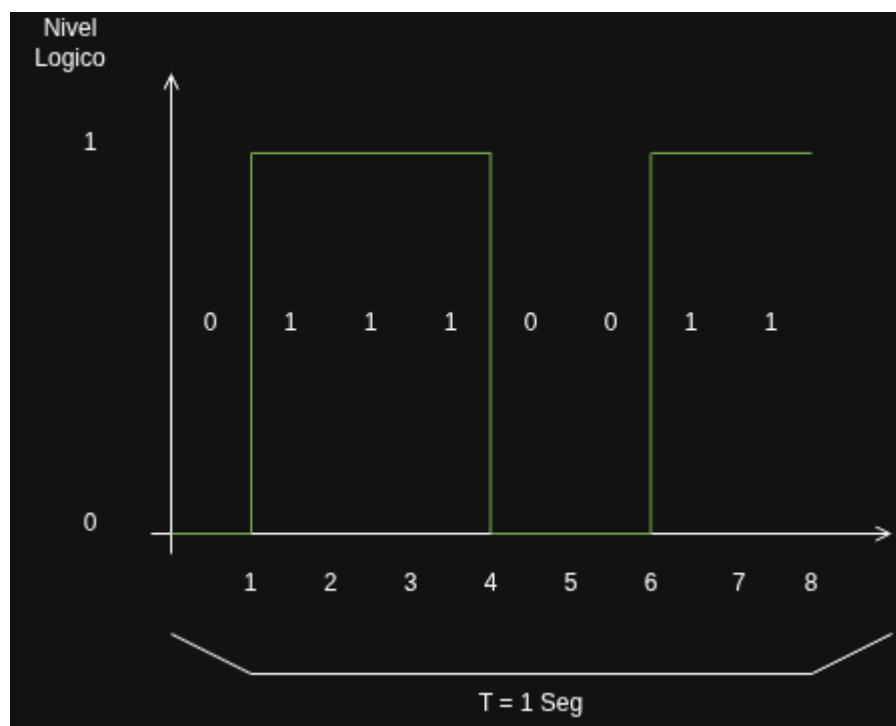


Figura 10. Forma de onda correspondiente a la Señal 2.

Gracias a este banco de pruebas fue posible verificar el funcionamiento integral del sistema, desde la captura de datos en los GPIO, pasando por la comunicación entre el espacio de kernel y el espacio de usuario, hasta la correcta representación gráfica de las señales en la interfaz web.

Automatización del proceso de carga y descarga del sistema completo

Durante la etapa de desarrollo y pruebas fue necesario cargar y descargar repetidamente todos los componentes del sistema, incluyendo el Device Tree Overlay, el módulo del kernel y la aplicación de usuario. Con el objetivo de simplificar este procedimiento, se desarrollaron dos scripts en Bash que automatizan completamente el despliegue y la desinstalación del sistema: `deploy.sh` y `undeploy.sh`.

El script `deploy.sh` se encarga de preparar el entorno de ejecución realizando de manera secuencial las siguientes tareas:

- Eliminar posibles configuraciones residuales provenientes de ejecuciones anteriores.
- Cargar el Device Tree Overlay (`my_overlay.dtb`) para habilitar la configuración de hardware necesaria.
- Insertar el módulo del kernel (`gpio_driver.ko`) mediante `insmod`.
- Ajustar los permisos del archivo de dispositivo `/dev/gpio_driver`, permitiendo que la aplicación de usuario pueda acceder al controlador sin restricciones adicionales.
- Activar automáticamente el entorno virtual de Python utilizado por la aplicación.
- Iniciar la aplicación de usuario (`app.py`), responsable de la adquisición y visualización de datos.

Por otro lado, el script `undeploy.sh` realiza el proceso inverso, liberando todos los recursos utilizados por el sistema. Entre sus funciones se encuentran la descarga del módulo del kernel y la eliminación de las configuraciones aplicadas durante el despliegue.

La utilización de estos scripts permitió reducir significativamente el tiempo necesario para iniciar y finalizar las pruebas, además de garantizar que todos los integrantes del grupo ejecutaran exactamente la misma secuencia de pasos. Esto mejoró la reproducibilidad de los ensayos, facilitó las tareas de depuración y minimizó errores humanos asociados a la configuración manual del sistema.

Resultados Obtenidos

1. Conexion del sistema:

En la *Figura 11* podemos ver el conexionado físico utilizado. Podemos observar los siguientes elementos:

- Raspberry PI Modelo 3B.
- Fuente de alimentación de 5v.
- Arduino UNO.
- 2 Divisores resistivos con resistencias de 3.3k y 6.8k.

Obs: El circuito con resistencias no es más que dos divisores resistivos para convertir la salida del arduino de **5v** a **3.3v**.

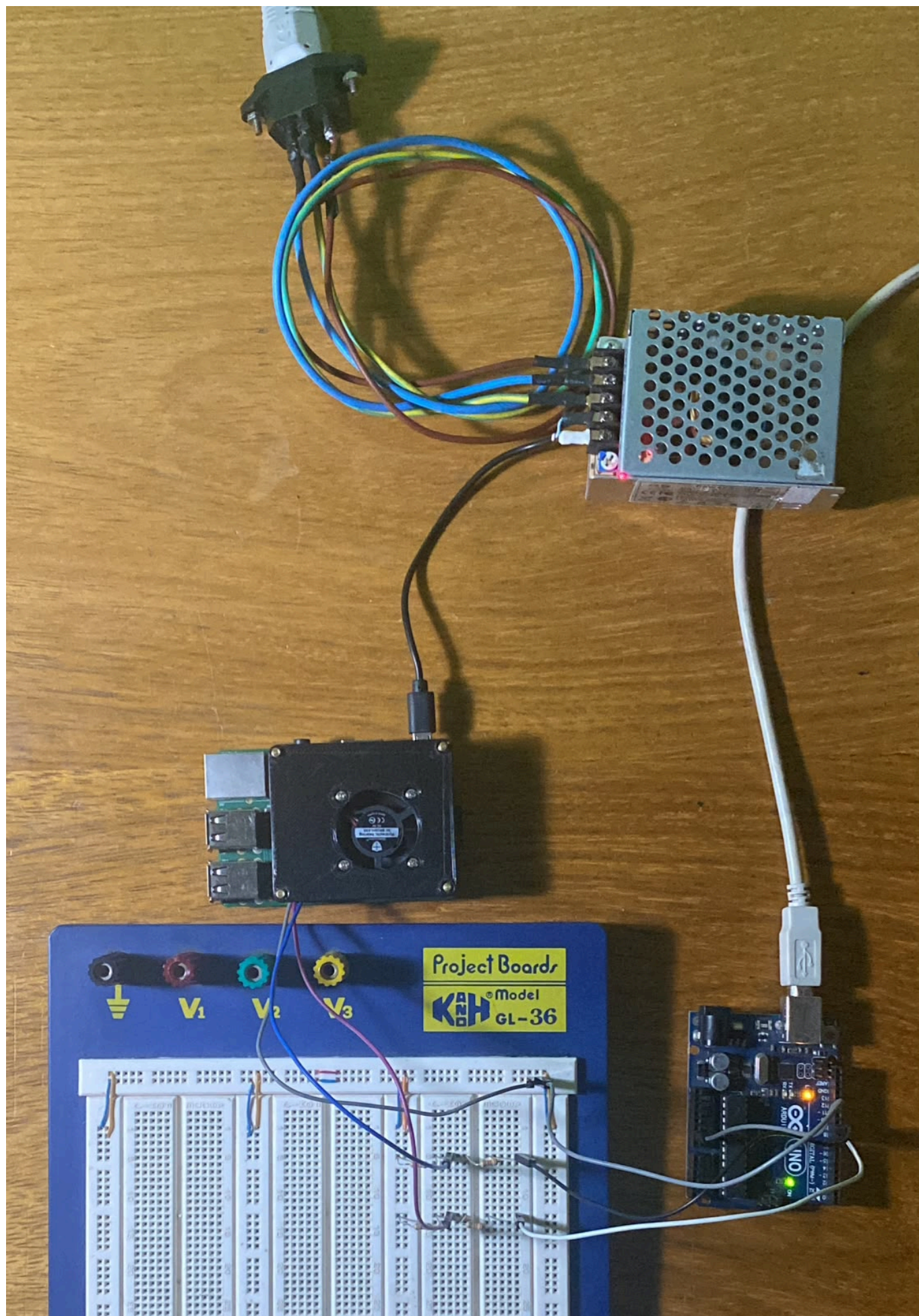
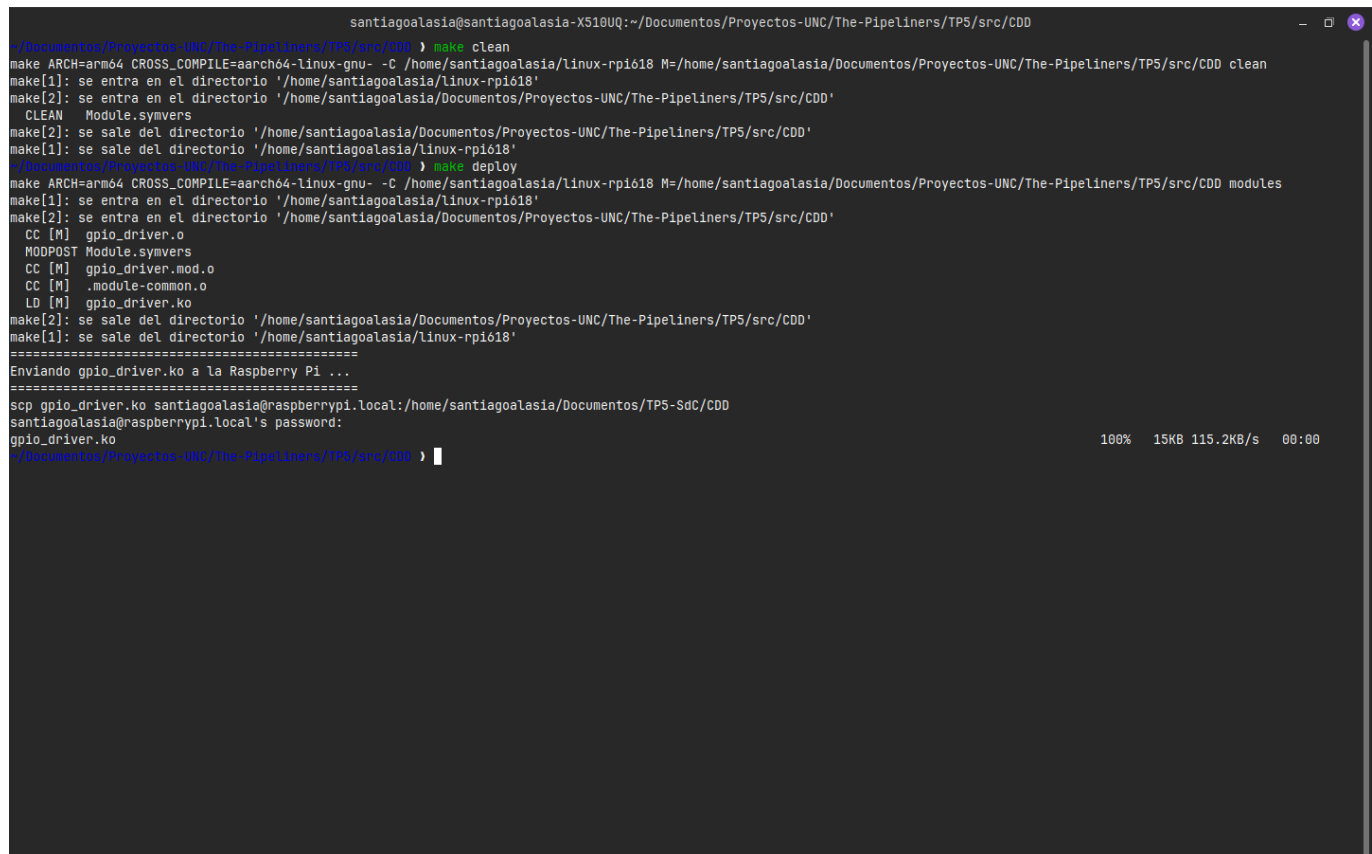


Figura 11. Conexión de la Raspberry PI con el arduino UNO

2. Compilación y envío:

```
# Limpieza de la compilación anterior
make clean

# Compila y envia el .ko a la raspberry pi
make deploy
```



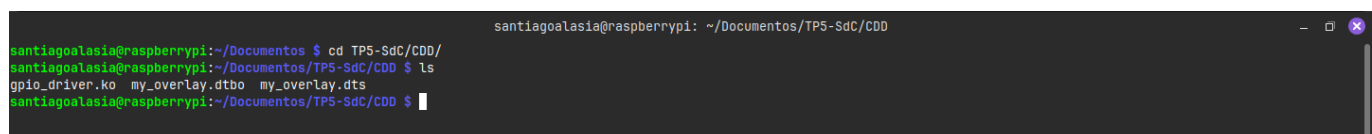
```
santiagoalasias@santiagoalasias-X510UQ:~/Documentos/Proyectos-UNC/The-Pipeline/TP5/src/CDD
~/Documentos/Proyectos-UNC/The-Pipeline/TP5/src/CDD ) make clean
make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- -C /home/santiagoalasias/linux-rpi618 M=/home/santiagoalasias/Documentos/Proyectos-UNC/The-Pipeline/TP5/src/CDD clean
make[1]: se entra en el directorio '/home/santiagoalasias/linux-rpi618'
make[2]: se entra en el directorio '/home/santiagoalasias/Documentos/Proyectos-UNC/The-Pipeline/TP5/src/CDD'
CLEAN Module.symvers
make[2]: se sale del directorio '/home/santiagoalasias/Documentos/Proyectos-UNC/The-Pipeline/TP5/src/CDD'
make[1]: se sale del directorio '/home/santiagoalasias/linux-rpi618'
~/Documentos/Proyectos-UNC/The-Pipeline/TP5/src/CDD ) make deploy
make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- -C /home/santiagoalasias/linux-rpi618 M=/home/santiagoalasias/Documentos/Proyectos-UNC/The-Pipeline/TP5/src/CDD modules
make[1]: se entra en el directorio '/home/santiagoalasias/linux-rpi618'
make[2]: se entra en el directorio '/home/santiagoalasias/Documentos/Proyectos-UNC/The-Pipeline/TP5/src/CDD'
CC [M] gpio_driver.o
MODPOST Module.symvers
CC [M] gpio_driver.mod.o
CC [M] .module-common.o
LD [M] gpio_driver.ko
make[2]: se sale del directorio '/home/santiagoalasias/Documentos/Proyectos-UNC/The-Pipeline/TP5/src/CDD'
make[1]: se sale del directorio '/home/santiagoalasias/linux-rpi618'
=====
Enviando gpio_driver.ko a la Raspberry Pi ...
=====
scp gpio_driver.ko santiagoalasias@raspberrypi.local:/home/santiagoalasias/Documentos/TP5-SdC/CDD
santiagoalasias@raspberrypi.local's password:
gpio_driver.ko
100% 15KB 115.2KB/s 00:00
~/Documentos/Proyectos-UNC/The-Pipeline/TP5/src/CDD )
```

Figura 12. Terminal PC Host, compilación y envío

3. Verificación de recibo:

```
# Nos movemos a la carpeta destino
cd TP5-SdC/CDD/

# Revisamos que se encuentre el archivo .ko
ls
```



```
santiagoalasias@raspberrypi: ~/Documentos/TP5-SdC/CDD
santiagoalasias@raspberrypi:~/Documentos/TP5-SdC/CDD$ cd TP5-SdC/CDD/
santiagoalasias@raspberrypi:~/Documentos/TP5-SdC/CDD$ ls
gpio_driver.ko my_overlay.dts my_overlay.dts
santiagoalasias@raspberrypi:~/Documentos/TP5-SdC/CDD$
```

Figura 13. Terminal Raspberry Pi, verificación

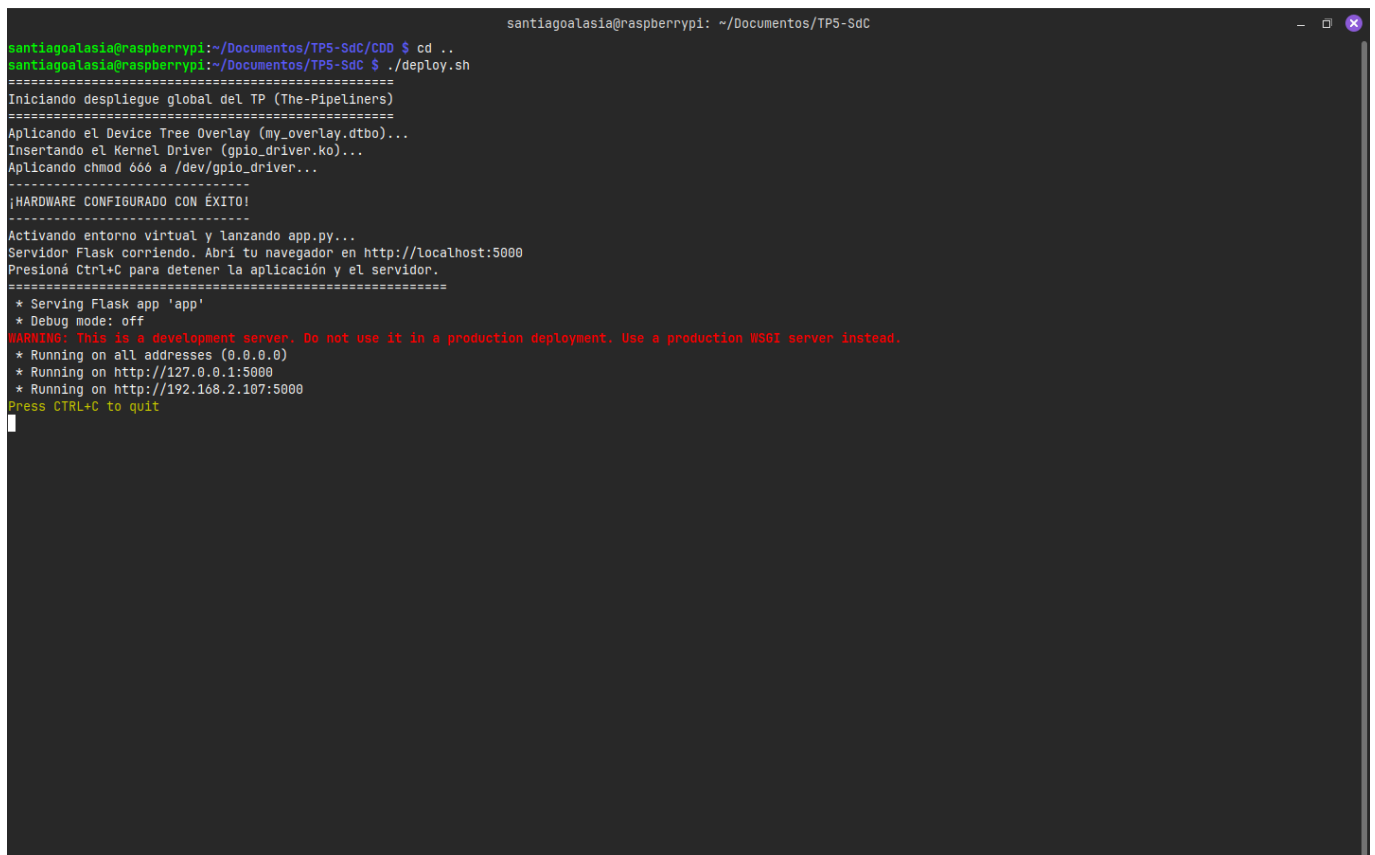
4. Inserción del modulo y Despliegue de la app


```
# Nos ubicamos en la carpeta del proyecto
cd ..

# Lanzamos el script de bash que creamos
./deploy.sh

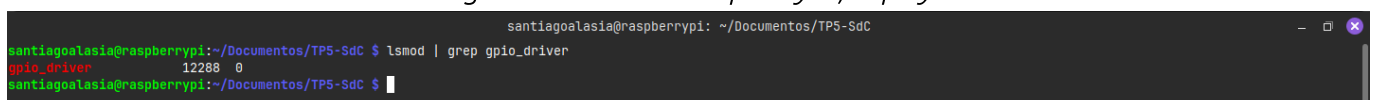
# Si accedemos desde el navegador a la ip y puerto seleccionado vamos a ver
nuestra sistema funcionando.

# Tambien podemos revisar el log de eventos y chequear el estado de nuestro
modulo
lsmod | grep gpio_driver
```



```
santiagoalasias@raspberrypi: ~/Documentos/TP5-SdC
santiagoalasias@raspberrypi:~/Documentos/TP5-SdC $ cd ..
santiagoalasias@raspberrypi:~/Documentos/TP5-SdC $ ./deploy.sh
=====
Iniciando despliegue global del TP (The-Pipelineers)
=====
Aplicando el Device Tree Overlay (my_overlay.dtbo)...
Insertando el Kernel Driver (gpio_driver.ko)...
Aplicando chmod 666 a /dev/gpio_driver...
=====
¡HARDWARE CONFIGURADO CON ÉXITO!
=====
Activando entorno virtual y lanzando app.py...
Servidor Flask corriendo. Abri tu navegador en http://localhost:5000
Presioná Ctrl+C para detener la aplicación y el servidor.
=====
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.2.107:5000
Press CTRL+C to quit
```

Figura 14. Terminal Raspberry Pi, deploy



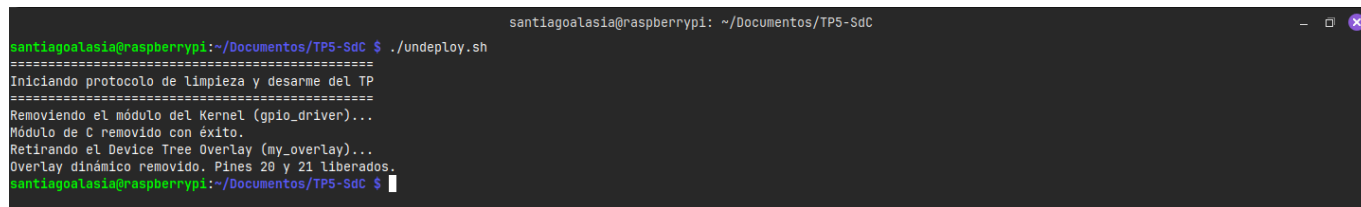
```
santiagoalasias@raspberrypi: ~/Documentos/TP5-SdC
santiagoalasias@raspberrypi:~/Documentos/TP5-SdC $ lsmod | grep gpio_driver
gpio_driver      12288  0
santiagoalasias@raspberrypi:~/Documentos/TP5-SdC $
```

Figura 15. Terminal Raspberry Pi, verificación modulo insertado

5. Eliminacion del Modulo y limpieza

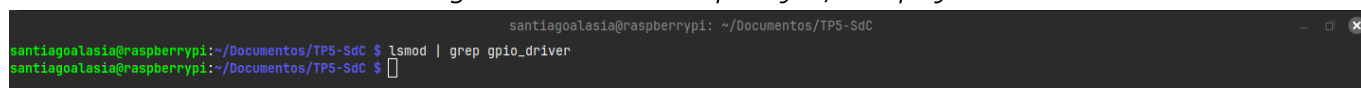
```
# Lanzamos el script de limpieza y verificamos que el modulo se haya
descargado
./undeploy.sh

lsmod | grep gpio_driver
```



```
santiagoalasia@raspberrypi: ~/Documentos/TP5-SdC
santiagoalasia@raspberrypi:~/Documentos/TP5-SdC $ ./undeploy.sh
=====
Iniciando protocolo de limpieza y desarme del TP
=====
Removiendo el módulo del Kernel (gpio_driver)...
Módulo de C removido con éxito.
Retirando el Device Tree Overlay (my_overlay)...
Overlay dinámico removido. Pines 20 y 21 liberados.
santiagoalasia@raspberrypi:~/Documentos/TP5-SdC $
```

Figura 16. Terminal Raspberry Pi, undeploy



```
santiagoalasia@raspberrypi: ~/Documentos/TP5-SdC
santiagoalasia@raspberrypi:~/Documentos/TP5-SdC $ lsmod | grep gpio_driver
santiagoalasia@raspberrypi:~/Documentos/TP5-SdC $
```

Figura 17. Terminal Raspberry Pi, verificación modulo eliminado

Conclusión general

El desarrollo e implementación de este sistema de adquisición permitió validar de manera práctica los conceptos de comunicación entre el **espacio de kernel** y el **espacio de usuario** bajo el sistema operativo Linux. Este diseño garantizó un acceso seguro, aislado y eficiente al hardware de la Raspberry Pi, evitando los riesgos de colisiones de memoria o bloqueos de pines comunes en las metodologías legadas.

Asimismo, la integración de un servidor intermedio en Python junto con un frontend desacoplado en Chart.js permito observar las señales muestreadas.

En conclusión, el proyecto cumple con los objetivos de estabilidad, modularidad y rendimiento definidos al principio del trabajo.

Bibliografía Consultada

- Implementation of Linux GPIO Device Driver on Raspberry Pi Platform - Vu Nguyen.
- Linux Driver Development with Raspberry Pi.
- Material de la catedra.
- Raspberry Pi Foundation, Linux Kernel Source Tree for Raspberry Pi. GitHub. Disponible en: <https://github.com/raspberrypi/linux>.
- Tutoriales de Johannes4Linux. Disponible en: https://github.com/Johannes4Linux/Linux_Driver_Tutorial
- Kernel Device Tree.